SEASUG Paper 71-2026

Bridging the Gap: Converting a Protocol-Level Analysis Program into a TA Template

Xiao Xiao, Merck & Co., Inc., Rahway, NJ, USA.

ABSTRACT

In a highly regulated clinical programming environment, standardized programming practices are essential for promoting quality, consistency, and reuse. Standard program templates provide a scalable approach that can be adapted across studies while maintaining alignment with company standards. This paper describes lessons learned from converting a protocol-level analysis program into a therapeutic area (TA) template and from enhancing reusable TA macros.

The discussion is organized around the macro development lifecycle: define, develop, validate, and execute. It highlights practical considerations in mock-up design, such as generalization across study designs, alignment with existing standards, and clear separation of study-specific and reusable elements. It also describes key development practices, including parameterization, standardized naming, data-driven logic, robust input validation, debugging support, and comprehensive testing. Finally, it emphasizes the importance of documentation, including intended use, expected inputs, limitations, and parameter definitions, to support sustainable adoption and maintenance.

Although the paper focused on a conversion effort, these foundational principles also apply to developing a TA template from scratch. By sharing these lessons learned, the paper provides practical guidance for programmers building reusable tools in regulated environments.

INTRODUCTION

Macros are an important part of clinical programming because they help organize code, reduce repetition, and support reuse and maintenance. In a regulated environment, macros also promote standardization by reducing unnecessary variation across programs and studies. However, developing reusable macros requires more than simply wrapping existing code. A macro intended for broader use must be designed with flexibility, robustness, traceability, and usability in mind.

This paper focuses on the transition from protocol-level analysis programs to TA templates. In this context, study-specific logic must be reviewed and, where possible, generalized so that the resulting macro can support multiple studies while remaining consistent with company standards. The development process typically involves iterative refinement of mock-ups, parameters, validation logic, and documentation. The discussion is organized around the main stages of macro development: define, develop, validate, and execute.

FROM PROTOCOL-LEVEL ANALYSIS PROGRAM TO TA TEMPLATE: KEY CONSIDERATIONS

Converting protocol-level analysis code into a TA template requires careful refactoring rather than simple reuse. Study-specific assumptions should be parameterized or replaced with more general logic, while balancing standardization, flexibility, robustness, traceability, and ease of use. The process is often iterative: user acceptance testing and post-deployment feedback may identify opportunities to refine the template and strengthen documentation.

Key considerations in this transition include:

- **Identify and parameterize study-specific assumptions** by replacing hard-coded values with macro parameters or dynamic logic to enhance automation and flexibility.
- **Use generalized logic** to support adaptivity and expand applicability across future studies, while targeting the common use case rather than all edge cases.

- **Standardize parameter naming and structure** to promote consistency, traceability, readability, and ease of use.
- **Implement robust input controls and validation checks** to strengthen quality control and reduce user error.
- **Apply thorough testing, validation, and user acceptance review** to confirm correctness, reproducibility, and usability.
- **Develop comprehensive documentation** to support usage, training and maintenance.

The following sections provide a stage-by-stage discussion of the macro development lifecycle and show how these considerations are applied in practice.

DEFINE STAGE

Before building a macro, it is essential to fully understand the analysis requirements so that appropriate logic can be designed to retrieve and validate inputs, process data, and generate results. At the define stage, statisticians provide these requirements through TA-level reporting mock-ups. The programmer should then carefully review the mock-up from a programming perspective. Key considerations include, but are not limited to, the following:

- **Clarity of the mock-up:** Are the table rows and columns clearly defined? Are titles, footnotes, and legends sufficient to explain the contents of the table body or graphical display, including any record-selection criteria, how numerators and denominators are defined, and how missing values are handled? Are the sorting order and color scheme specified?
- **Accuracy and completeness:** Does the mock-up align with the analysis requirements and include all intended elements? For example, are total counts included, are percentages calculated, and are the relevant statistical model outputs displayed, and are strategies for handling special issues, such as convergence failure, clearly specified?
- **Generalizability:** Is the mock-up sufficiently general to meet the analysis needs of most studies? Following the 80/20 rule, standard macros should ideally address the majority of expected use cases. Common study design scenarios, such as open-label single-arm, randomized, parallel, and crossover, should be considered. Study-specific and variable elements should be distinguished from fixed elements so the corresponding macro parameters can be designed to control output generation.
- **Consistency with prior conventions:** Does the format align with established reporting practices, including terminology, indentation, sorting order, and line breaks?
- **Readability and programming feasibility:** Is the display practical to implement? If a table is too dense or a graph has limited space, alternative display options should be considered.

A well-defined mock-up is a critical prerequisite for macro development, as it serves as the design specification for implementation. The TA version of the mock-up should include detailed annotations for data retrieval, such as variable sourcing, filtering rules, and additional analysis logic not evident from titles and footnotes alone. These annotations support accurate implementation and efficient programming during macro development and validation, and they are incorporated into the user manual, which serves as the primary reference for users to understand the macro's definition, intended purpose, and proper use.

Because analysis outputs are based on ADaM datasets, the available data must be assessed before developing the reporting macro. If required information is not available in ADaM, the macro developer discusses with the statistician and ADaM programmer whether it should be added as a parameter, derived as a variable, or handled within the macro. Required analysis data are ideally available in ADaM to support the "analysis-ready and one-proc-away" principle. When multiple ADaM versions may exist, backward compatibility must be considered during development. If compatibility cannot be achieved, the user manual clearly documents the limitation and any related user instructions. Supporting variables needed for formatting or sorting may be generated within the macro in an intermediate dataset.

DEVELOP STAGE

Once the mock-up has been established, programmers can proceed with macro development. For a protocol-level analysis program, the existing analysis code should be compared with standard source data and the TA mock-up to identify study-specific assumptions and assess code reuse. Whether generalizing a protocol-level analysis or building a TA macro from scratch, study-specific elements should be parameterized rather than hard-coded.

PARAMETERIZATION OF STUDY-SPECIFIC ELEMENTS

Typical macro parameters may cover the following aspects:

- **Input data and selection criteria:** Parameters specify input datasets and record-selection conditions.
- **Analysis grouping variables:** Parameters define treatment or indication groups for by-group summaries and to specify whether a total group should be created and included.
- **Method or analysis-type variables:** These parameters apply when the same macro supports multiple analysis algorithms or output variants that differ in processing logic, display, or focus, but are not sufficiently distinct to justify separate macros from a maintenance perspective.
- **Display-related variables:** These include parameters that define sorting rules, display order, row grouping, indentation, and formatting specifications used to control how data are presented in the table or figure.
- **Reporting-related variables:** These include titles, footnotes, column headers, data source citations, and page and layout settings such as page size, orientation, overflow control, and fonts.

After the parameter list has been defined, parameter names follow company conventions and remain descriptive and consistent. When standard parameter names are available, they should be reused rather than creating new ones unnecessarily. This improves consistency across macros and makes them easier for users and maintainers to understand.

TA MACRO DESIGN CONSIDERATIONS

With the mock-up and parameters established, TA macro development should focus on implementation quality. Existing logic may be reused where appropriate, provided it is refined to align with the TA mock-up and company data standards. Accordingly, TA-level macros should be designed with four key qualities in mind: flexibility, robustness, dynamic behavior, and debugging support.

Flexibility

Hard-coded logic should be minimized by using parameters to accommodate study-specific analysis and output needs, such as population selection, source data, grouping variables, formats, sorting order, and page orientation. However, parameters should be used judiciously to avoid over-parameterization and unnecessary complexity.

Below is an example macro call that passes a subject-level population dataset through the population_from parameter and optional selection criteria through the population_where parameter:

```
%macro_name(
    population_from=lptda.adsl,
    population_where=%str(and saffl='Y'));
```

The macro then reads the population dataset internally as follows:

```
proc sql;
     create table _population_extract as
         select distinct
```

```
            1 as _studyid
            ,a.*
        from
            &population_from as a
        where
            1
            &population_where.
    ;
quit;
```

Robustness

To ensure the macro behaves reliably, predictably, and safely across expected and unexpected inputs, parameter values should be validated and normalized when necessary. A structured validation framework is recommended, especially when parameters have dependencies, restricted values, or severity-based handling rules. Figure 1 shows an example decision tree for input macro parameter validation. Depending on the severity of an invalid parameter, the macro may either abort with an error message or revert to a default setting and continue. Default values should be assigned when a standard value is available or when a setting applies to most scenarios, supporting consistent execution. Internal quality-control checks may be applied where appropriate to help ensure reliable execution.

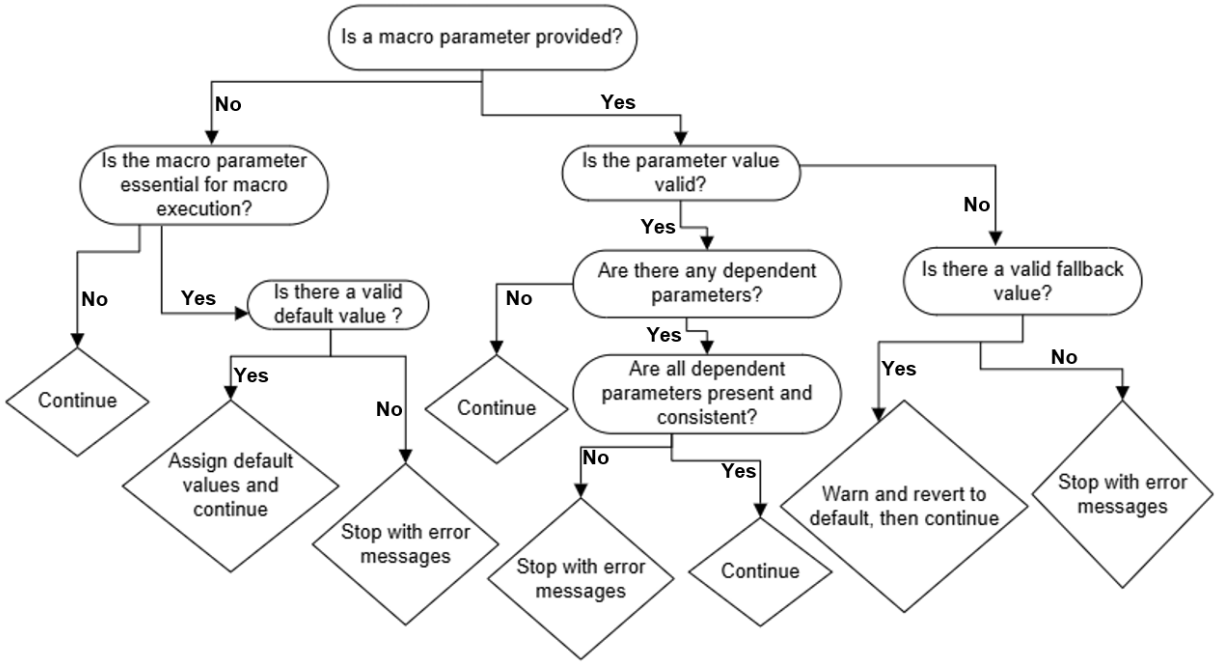


Figure 1. Decision tree for validating input macro parameters

The following example illustrates basic validation and default handling for a macro parameter:

```
%* Check that DISPLAY_TOTALS has a valid Y/N value;
%if %upcase(&display_totals) ne Y and %upcase(&display_totals) ne N %then
    %do;
        %put ============Usage for Macro &sysmacroname=====================;
        %put &war1&war2: Invalid value for DISPLAY_TOTALS: [&display_totals];
        %put &war1&war2: Valid values for DISPLAY_TOTALS are Y or N;
```

```
        %put &war1&war2: Defaulting DISPLAY_TOTALS to Y and continuing
execution;
        %put ============Usage for Macro &sysmacroname=====================;
        %let display_totals = Y;
    %end;
```

Dynamic behavior

Dynamic behavior in a TA standard template should encompass both conditional branching and data-driven automation. The macro should follow clearly defined execution paths based on input conditions so that processing remains controlled, predictable, and aligned with the intended analysis logic. Where appropriate, the macro should also adapt automatically to the input data to improve robustness, user-friendliness, and consistency, while remaining traceable through log messages and reviewable through documentation and code comments. This may include automatically adjusting graph axis ranges, label lengths, or page overflow control as needed. In this way, dynamic behavior supports both reliable execution and a seamless user experience.

The following example illustrates dynamic setting of the x-axis range:

```
      proc sql noprint;
          /* Derive tick values and axis limits for the current parameter */
          select distinct xvarn into :x_tick_values_&i separated by ' '
              from __final
              where upcase(paramcd) = "%upcase(&current_paramcd)"
                and not missing(xvarn)
              order by xvarn;

          select min(xvarn), max(xvarn)
              into :x_min_&i trimmed, :x_max_&i trimmed
              from __final
              where upcase(paramcd) = "%upcase(&current_paramcd)"
                and not missing(xvarn);
      quit;
```

Debugging support

Debugging support should be built into the macro through a modular code structure, a clear programming workflow, section header comments, informative log messages, and retained intermediate datasets where useful. Log messages should identify the relevant step or function, report key parameter values when helpful, and explain warnings, errors, defaults, fallback actions, and selected branches. Intermediate datasets should be preserved to support inspection of derived results during development and review. Validation issues should be reported clearly in the log with the expected result, observed value, and required action so that problems can be identified quickly during troubleshooting. Together, these practices support efficient review, troubleshooting, and maintenance.

Illustrative Case Study: Refactoring in Practice

A brief example illustrates these principles in practice. Table 1 summarizes representative changes made when converting a patient-reported outcomes (PRO) disposition table from a protocol-level implementation to a TA macro. The original program was developed under tight timelines in response to an FDA information request, when building a reusable TA solution was not feasible. The conversion was completed afterward as part of a broader TA standardization effort.

Development Consideration	Protocol Level (Before)	TA Level (After)
Parameterization	Categories, titles and display elements hard-coded with study-specific values	Customized through descriptive parameters controlling categories, display text and reporting options
Naming conventions	Abbreviated, study-specific parameter names	Standardized, descriptive names consistent with TA library conventions
Input validation	None; Assume data are present with the expected structure.	Comprehensive checks including dataset existence, informat validity, and cross-parameter consistency; invalid inputs abort with clear messages or revert to documented defaults
Dynamic behavior	Fixed display structure; zero-count categories excluded	Display structure generated dynamically from parameter settings and data characteristics, with support for complete category presentation and optional user control of layout
Count exclusion logic	Mutual exclusivity implied by NODUPKEY sort order; no explicit cross-group exclusion check	Explicit cross-group exclusion enforced through generalized logic; confirmed as an analytical refinement at the TA level
No-data handling	None	Record count checked after data extraction; graceful fallback to a "No data to report" table
Debugging support	Binary flag controls only dataset deletion	Three-level system supporting clean run, diagnostic inspection, and resolved code generation for submission review
Traceability	None	Applied parameter values saved to a permanent dataset

Table 1. Selected Design Changes from a Protocol-Level Program to a TA Macro

As shown in Table 1, most changes focused on structure rather than substance, wrapping the existing analysis logic in a more robust, flexible, and traceable framework suitable for reuse across studies. However, the conversion also presented an opportunity to revisit and strengthen the underlying analysis logic, as illustrated by the count exclusion row. This serves as a reminder that conversion is best approached as a deliberate review process, and that close collaboration with the statistician remains valuable throughout. These development decisions then inform the formal testing and validation activities discussed next.

DEVELOPER TESTING AND USER ACCEPTANCE TESTING

Developer testing should be conducted early and iteratively throughout development to confirm that the macro is correct, robust, and aligned with the specification. Although protocol-level macros also require correct implementation, they do not typically undergo the same level of developer testing as TA-level macros. The following checks should address functionality, output quality, error handling, reproducibility, and submission readiness:

1. **Accuracy and generality testing:** Verify that the macro produces correct results according to the mock-up and performs as expected across representative studies or scenarios. Where needed, dummy data may be created to broaden test coverage before handoff to user acceptance testing, including long treatment descriptions, intended population flags, and other relevant test inputs.
2. **Functional testing and clean logs:** Check that parameters behave as intended, that default settings conform to the specification, and that logs contain no errors or avoidable warnings.
3. **Invalid-parameter and edge-case testing:** Ensure that invalid inputs are handled as specified, whether by rejection or fallback behavior, and that unusual valid inputs are processed consistently, with diagnostic messages generated as needed for debugging.
4. **Code-resolution and consistency testing:** Confirm that the resolved open code is suitable for submission and remains consistent with the macro call output, with only minor edits permitted that do not affect the analysis logic or results. This also includes checking for obsolete or unused parameters introduced during conversion or subsequent development changes, which should be removed where appropriate.
5. **Reproducibility testing**: Establish that repeated runs with the same inputs produce the same results.
6. **Documentation synchronization**: Ensure that changes in logic, outputs, assumptions, or usage instructions are reflected in the supporting documentation.

A temporary test-mode control may be inserted at key modules to support incremental testing by allowing execution to stop at predefined steps. This enables intermediate outputs to be reviewed and verified before later processing continues, thereby facilitating earlier issue detection and more efficient defect isolation. Such controls should be used only during development and should be disabled before production release.

An example of a temporary test-mode control is shown below:

```
%if %symexist(TEST_STOP_STEP) %then %do;

    %if &TEST_STOP_STEP = 6 %then %do;

        %put NOTE: Test mode ╥ step 6 stopping after summary
generation;

        %goto exit;

    %end;

%end;
```

Tests should be documented thoroughly and consistently to provide evidence of evaluation activities and to support long-term maintenance. Clear documentation also facilitates validation, particularly when multiple test cases and parameter settings are involved, by allowing QC reviewers to understand exactly what was tested and to verify that the results are valid and reproducible through alternative coding approaches. Recommended documentation includes the test case identifier, objective and design rationale, input data, parameter settings, and the output files generated.

Once developer testing is complete, the macro may be distributed for user acceptance testing. UAT, conducted in real-study scenarios with intended end users or representative reviewers where applicable, may provide feedback on usability, workflow fit, and documentation needs. This feedback can help identify training needs, guide coding refinement, assess documentation adequacy, and support agreement on the scope and timeline for requested changes before production release.

DOCUMENTATION AND TRACEABILITY

Documentation and traceability expectations apply to both protocol-level macros and TA macros, although TA macros typically require a more formal and detailed presentation because they are more complex and intended for a broader range of users and reviewers. Documentation operates at two levels: an internal, code-focused layer within the macro and an external, user-facing layer that is particularly important for TA macros and provides comprehensive guidance on how the macro is defined and how it should be used.

The internal documentation should support development, review, and maintenance. For protocol-level macros, brief parameter comments at the macro definition may be sufficient, whereas TA macros should include clearer and more detailed parameter descriptions in the program header. In both cases, the macro purpose, valid parameter values, default values where applicable, key dependencies, underlying assumptions, and usage constraints should be documented. For TA macros, a parameter dataset should also be created to record applied parameter values for traceability, and documentation should be version-controlled to support ongoing maintenance and updates.

The user-facing macro specification or user manual serves as the primary supporting document and should be coauthored by the statistician and the developer programmer to provide a practical and comprehensive reference for users. As noted earlier in the define-stage section, the document describes the analysis intent, design rationale, underlying assumptions, mock-up context, input data versions and structures, and processing logic. It also includes parameter descriptions, macro call examples, and practical execution guidance such as usage limits and recommended settings. As the macro is refined, the user manual should be kept in sync with the code, and its revision history should record change requests, the nature of the changes made, and the corresponding rationale, thereby preserving the history of macro development and maintenance.

VALIDATE STAGE

Validation of a TA macro should be performed as an independent end-to-end review. Once development has stabilized, the developer and validator should agree on the validation strategy, including the full scope of testing, to ensure that validation is as complete and robust as possible across all valid execution paths and representative test cases. For a first-version macro, double programming is required, in which the validator independently codes the program based on the mock-up and the same input data, without reference to the macro being validated, and confirms that the results are reproducible. The validator should also confirm that final outputs are dated on or after the macro program timestamp. In addition, the validator should manually review outputs for elements that may not be fully captured by data comparison alone, such as formatting, titles, and footnotes. Additional testing or stakeholder review may be performed when needed.

Following output verification, a code review should confirm compliance with good programming practices and adequate documentation. Validation should also confirm that invalid inputs generate the expected messages and logs are free of errors and avoidable warnings. Any exceptions that do not affect macro use should be documented in the validation tracker. Test cases, inputs, parameter settings, and outputs should be recorded for traceability, and QC evidence should be generated only after the macro outputs have been produced.

EXECUTION STAGE

Macro execution may include user training and support as needed to promote correct and consistent use. During routine use, user feedback should be collected to identify clarity issues, documentation gaps, and opportunities for improved usability. Depending on the nature of the issue, the response may include

training, a revised FAQ list, user manual updates, macro enhancement, or deferral of the requested change to the protocol macro when study-level customization is more appropriate. Any updates should follow the macro development and QC cycle, with steps applied as appropriate, before production release.

Ongoing use of the macro should be supported by:

- clear and up-to-date documentation;
- consistent version control;
- an established process for issue reporting, triage, resolution tracking, and installation in the TA environment.

CONCLUSION

This paper reviewed the transition from protocol-level analysis programs to TA templates and highlighted practical considerations across the TA macro lifecycle. The discussion showed that successful standardization requires more than reuse of existing code. It requires deliberate design choices to separate study-specific from reusable logic, incorporate data-driven and dynamic behavior where appropriate, and support consistent, reliable execution and long-term maintainability across studies.

Although the paper focused on a conversion effort, the same principles also apply to developing TA templates from scratch. Whether developed from legacy protocol code or built from scratch, successful TA templates rely on the same fundamentals: clear specifications, disciplined parameterization, robust validation, and comprehensive documentation that supports both use and long-term maintenance. Applied thoughtfully, these principles help create sustainable macro solutions that improve quality, consistency, and efficiency in regulated clinical programming environments.

ACKNOWLEDGMENTS

The author sincerely thanks Hong Qi, Mary Varughese, and Amy Gillespie for their constructive suggestions and support. The author also acknowledges the use of the OpenAI enterprise version, ChatGPT (GPT-5.4 mini mode), via GPTeal, to assist with brainstorming, organizing ideas, refining structure, and improving clarity and word choice.

RECOMMENDED READING

- *Carpenter's Complete Guide to the SAS Macro Language*

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the author at:

Xiao Xiao

Merck & Co., Inc., Rahway, NJ, USA

xiao.xiao3@merck.com